

Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Tecnologias de Sistemas Operativos

Ano Letivo de 2025/2026

Sistema de Ficheiros com Deduplicação

Grupo 6

João Teixeira
A106836

Miguel Barbosa
A104451

Miguel Barrocas
A104272

May 15, 2026

TSO

Índice

1. Introdução 1

2. Arquitetura e Motor de Deduplicação 1

2.1. Desenho do Sistema 1

2.1.1. Separação entre Tamanho Lógico e Físico 1

2.1.2. Contentores de Armazenamento 2

2.2. Deduplicação Inline 2

2.2.1. Interceção de Escritas 2

2.2.2. Cálculo de Hash SHA-256 3

2.2.3. Deteção de blocos duplicados 3

3. Gestão de Estado e Recuperação de Espaço 3

3.1. Estruturas de Dados 3

3.1.1. Hash Tables 3

3.2. Persistência e *Garbage Collection* 4

3.2.1. Modelo de Persistência 4

3.2.2. Sistema de Referências e Garbage Collection 5

3.2.3. Reutilização de Espaço com a `free_list` 5

3.2.4. Limitações do Modelo 5

4. Testes e Avaliação Experimental 5

4.1. Metodologia & Observabilidade (eBPF) 6

4.1.1. Ferramentas de Geração de Carga (FIO) 6

4.1.2. Observabilidade no Kernel (eBPF) 6

4.1.3. Automatização dos Testes 6

4.2. Resultados e Comparação 6

4.2.1. Baseline (FUSE Passthrough) 7

4.2.2. Cenários de Teste e Análise de Dados 7

4.2.3. Apresentação dos Resultados 8

5. Conclusão 9

6. Referências Bibliográficas 10

Lista de Figuras

Figura 1 Comparação de Desempenho: Overhead do Programa (W1), Impacto de Leitura (W2) e Ganhos em Stress Test (W3). 8

Figura 2 Flamegraph que destaca o custo criptográfico e o overhead de comunicação FUSE (receção e resposta de I/O). 9

Lista de Tabelas

Tabela 1 Ficheiros de metadados persistidos em `.meta/` 4
Tabela 2 Resumo das Médias de Desempenho (30 Iterações) entre dedup_fs e passthrough_fs 9

1. Introdução

Este projeto implementa um sistema de ficheiros baseado em **FUSE** (Filesystem in Userspace) que integra **deduplicação** *inline* a nível de blocos de 4KB. O sistema foi desenvolvido com o objetivo de servir como arquivo de dados, que cumpre um padrão de escrita do tipo **append--only**, onde cada ficheiro é atualizado sequencialmente e nunca é reescrito. Esta restrição simplifica a lógica de deduplicação, o que permite uma implementação eficiente e previsível.

Os objetivos principais deste projeto são:

1. Implementar um motor de deduplicação que detete blocos de dados duplicados e os elimine com recurso a deduplicação;
2. Manter transparência para o utilizador, com a apresentação de tamanhos lógicos corretos independentemente da partilha de blocos;
3. Garantir que os dados persistem entre montagens e desmontagens do sistema de ficheiros;
4. Avaliar experimentalmente a eficácia da técnica através de comparação com um *baseline* FUSE passthrough sem deduplicação.

2. Arquitetura e Motor de Deduplicação

O sistema de ficheiros implementado segue uma arquitetura modular que separa claramente a representação lógica, vista pelo utilizador, da representação física de armazenamento em disco. Esta secção detalha o desenho arquitetónico do sistema e explica como o motor de deduplicação funciona no caminho crítico das operações de escrita.

2.1. Desenho do Sistema

2.1.1. Separação entre Tamanho Lógico e Físico

A contribuição fundamental desta arquitetura é a clara separação entre o tamanho lógico e o tamanho físico dos ficheiros. O tamanho lógico corresponde ao tamanho real que o utilizador observa quando consulta as propriedades de um ficheiro através do comando `ls` ou chamadas ao sistema como `stat()`. Este tamanho permanece invariável, independentemente de quantos ficheiros partilham blocos de conteúdo idêntico através da deduplicação.

Por outro lado, o tamanho físico refere-se à quantidade real de espaço em disco ocupado pelos blocos únicos após a aplicação da deduplicação. Quando múltiplos ficheiros partilham o mesmo conteúdo em blocos de 4KB, apenas uma cópia física é armazenada em disco. O sistema mantém um contador de referências para cada bloco único, o que permite saber quantos ficheiros dependem daquele bloco. Quando um ficheiro que contém esse bloco é eliminado, o contador de referências é decrementado, e quando atinge zero, o espaço físico é recuperado para reutilização.

2.1.2. Contentores de Armazenamento

Os blocos únicos são armazenados fisicamente em ficheiros chamados contentores. Cada contentor é um ficheiro com um tamanho máximo de 1GB. Esta escolha de limite foi feita para equilibrar entre a complexidade de gestão de muitos ficheiros pequenos e a possibilidade de ficheiros muito grandes que possam dificultar operações de cópia ou manutenção. Do ponto de vista da persistência e segurança dos dados, a divisão em contentores de 1GB oferece vantagens cruciais. Primeiramente, isola o impacto de uma eventual corrupção de dados a nível do hardware (como um setor danificado no disco), o que confina a perda a um único contentor em vez de comprometer um arquivo monolítico massivo. Em segundo lugar, facilita imensamente as operações de backup e sincronização (por exemplo, através de ferramentas como o `rsync`), uma vez que o sistema apenas necessita de transferir os contentores novos ou recentemente modificados, o que torna a replicação de segurança muito mais rápida e eficiente.

Quando o sistema precisa de escrever um novo bloco único em disco, utiliza um algoritmo de alocação que funciona da seguinte forma. Primeiro, verifica se existe algum *slot* livre na `free_list`, que é uma lista de espaços que foram libertados quando ficheiros foram eliminados. Se existirem *slots* livres, o sistema reutiliza-os e escreve o novo bloco num contentor anterior. Isto reduz a fragmentação e maximiza a utilização do espaço em disco. Se não houver *slots* livres, o sistema escreve o bloco no contentor ativo, que é o contentor para o qual as escritas mais recentes foram dirigidas. Cada contentor ativo tem um file *descriptor* aberto para evitar *overhead* de abertura e fecho de ficheiros.

A localização física de cada bloco no contentor é rastreada através da estrutura `ChunkInfo`, que será descrita em detalhe na próxima secção.

2.2. Deduplicação Inline

2.2.1. Interceção de Escritas

A deduplicação *inline* significa que o processo de deteção de blocos duplicados ocorre no caminho crítico das operações de escrita, antes dos dados serem persistidos em disco. Esta abordagem contrasta com estratégias *offline*, onde os dados seriam inicialmente armazenados sem qualquer processamento e posteriormente um componente separado analisaria o armazenamento para eliminar redundâncias.

Quando uma aplicação de utilizador executa operações de escrita, o sistema interceta estas operações no nível do FUSE. Neste ponto, antes de qualquer dado ser persistido, o motor de deduplicação analisa o conteúdo a ser escrito. Este momento de interceção é crítico porque determina o destino dos dados e as referências que serão mantidas nos índices.

A estratégia de *inline* tem vantagens e desvantagens. Por um lado, garante que apenas dados únicos são escritos em disco, minimizando a *overhead* de armazenamento físico no ponto de inserção. Por outro lado, adiciona latência às operações de escrita, uma vez que o cálculo de *hashes* e a consulta aos índices de metadados precisam de ser completados antes da operação ser considerada concluída.

O sistema assume que as operações de escrita seguem um padrão *append-only*. Isto significa que um ficheiro nunca é modificado no meio, apenas estendido no final. Esta restrição simplifica significativamente a gestão de metadados e torna a validação do *offset* de escrita uma operação trivial. O sistema pode simplesmente verificar se o *offset* corresponde ao final atual do ficheiro;

se não corresponder, a operação é rejeitada. Esta propriedade é fundamental para manter a integridade do mapeamento entre a receita (*block_hashes*) e o conteúdo físico real em disco.

2.2.2. Cálculo de Hash SHA-256

A identificação de blocos duplicados na arquitetura é feito através da função SHA-256, que produz um valor de 256 bits (32 bytes) como resumo de qualquer sequência de entrada.

A propriedade crucial de uma função de *hash* criptográfica é que é determinística e tem excelente dispersão. Isto significa que o mesmo conteúdo de 4KB produz sempre o mesmo *hash*, enquanto conteúdos diferentes produzirão *hashes* com baixa probabilidade de colisão. Esta propriedade permite ao sistema usar o *hash* como um identificador único e confiável para cada bloco de conteúdo.

O *hash* SHA-256 é representado internamente como uma cadeia hexadecimal de 64 caracteres (mais um terminador nulo). Esta representação facilita o armazenamento em estruturas de dados indexáveis e a comparação entre *hashes*, uma vez que comparações de cadeias são mais diretas do que manipulação de valores binários de 256 bits.

2.2.3. Deteção de blocos duplicados

Após o cálculo do *hash*, a arquitetura consulta o índice `chunk_index` para determinar se este bloco já foi visto anteriormente. Quando um bloco é encontrado no índice (que indica uma duplicação), em vez de escrever os bytes novamente em disco, o sistema apenas incrementa o contador de referências do bloco existente. Isto comunica ao sistema que mais um ficheiro agora depende deste bloco. Nenhuma transferência de dados é necessária, apenas uma operação de escrita de metadados.

Quando um bloco é novo (não encontrado no índice), o sistema escreve-o em disco e cria uma nova entrada no índice com o contador de referências inicializado a 1. Este novo bloco está agora disponível para ser referenciado por ficheiros futuros.os.s.os.s.

3. Gestão de Estado e Recuperação de Espaço

3.1. Estruturas de Dados

O estado global do sistema de ficheiros é mantido em memória numa *struct*: `Context`, definida em `include/dedup_fs/metadata.h`. Esta estrutura agrega dois índices principais implementados com *Hash Tables*, uma lista de espaço livre e contadores globais de estatísticas.

3.1.1. Hash Tables

A escolha de *Hash Tables* para os dois índices principais justifica-se pela necessidade de pesquisas durante operações críticas como escrita e leitura. O sistema utiliza dois índices distintos com papéis complementares.

O `metaindex` mapeia o caminho de cada ficheiro (por exemplo da `"/pasta/ficheiro.txt"`) para a estrutura `FileMeta` correspondente. É consultado em praticamente todas as operações FUSE — `getattr`, `read`, `write`, `unlink`, entre outras. A chave é uma string com o caminho do ficheiro e o valor é um apontador para `FileMeta`:

```
typedef struct {
    EntryType type;         // TYPE_FILE ou TYPE_DIRECTORY
    GArray*   block_hashes; // Lista ordenada de hashes SHA-256 dos blocos
    size_t    logical_size; // Tamanho lógico do ficheiro em bytes
    mode_t    mode;         // Permissões e modo dos ficheiros/diretorias
} FileMeta;
```

O campo `block_hashes` é um `GArray` de strings onde cada posição `i` contém o *hash* SHA-256 do bloco `i` do ficheiro (blocos de 4 KB fixos). Este *array* serve para reconstruir os dados, o sistema percorre-o sequencialmente, localiza cada bloco no segundo índice e lê-o do "container" físico.

O `chunk_index` mapeia cada *hash* SHA-256 para a estrutura `ChunkInfo`, que descreve onde o bloco está armazenado e quantas vezes é referenciado:

```
typedef struct {
    char    hash[65];     // Hash SHA-256 em hexadecimal
    guint32 container_id; // ID do ficheiro container físico
    guint64 offset;       // Offset em bytes dentro do container
    guint32 ref_count;    // Número de referências ativas
} ChunkInfo;
```

Aqui está a estrutura da deduplicação. Quando um bloco é escrito, o seu *hash* é calculado e pesquisado no `chunk_index`. Se já existir, nenhuma escrita física ocorre — apenas o `ref_count` é incrementado.

3.2. Persistência e *Garbage Collection*

3.2.1. Modelo de Persistência

O sistema adota um modelo de persistência *lazy*: os metadados são mantidos exclusivamente em memória durante a execução e serializados para disco depois, através da sequência `dedup_destroy()` → `save_metadata()`. No arranque, `dedup_init()` → `load_metadata()` restaura o estado completo a partir dos ficheiros guardados. Se não existirem ficheiros de metadados (primeira montagem), o sistema arranca com estado vazio sem qualquer erro.

Os metadados são guardados em quatro ficheiros binários em `.meta/` dentro do `root_dir`:

Ficheiro	Conteúdo
`metaindex.bin`	Todos os `FileMeta` com caminhos e arrays de hashes
`chunk_index.bin`	Todos os `ChunkInfo` com localização e `ref_count`
`free_list.bin`	Todos os `FreeSlot` disponíveis para reutilização
`stats.bin`	Contadores globais e ID do container ativo

Tabela 1: Ficheiros de metadados persistidos em `.meta/`

A função `save_metadata()` escreve os metadados em ficheiros binários na pasta `meta/`. Para o *metaindex*, cada entrada começa pelo tamanho e conteúdo do caminho do ficheiro, seguido das suas propriedades e da lista de *hashes* dos blocos. O `chunk_index` e a `free_list` são mais

simples — como as suas estruturas não contêm apontadores, são escritas diretamente para disco. Quando o sistema é montado novamente, `load_metadata()` lê estes ficheiros pela mesma ordem e reconstroi o estado em memória.

3.2.2. Sistema de Referências e Garbage Collection

Cada bloco físico tem um contador `ref_count` no seu `ChunkInfo` que indica quantos ficheiros o estão a usar naquele momento. Quando um ficheiro é escrito, `engine_process_write()` divide os dados em blocos de 4 KB e calcula o *hash* SHA-256 de cada um. Se o *hash* já existir no `chunk_index`, o bloco já está guardado em disco — o `ref_count` é incrementado e não há qualquer escrita física. Se for um bloco novo, é escrito via `chunk_store_write()` e registado no `chunk_index` com `ref_count` = 1. Em ambos os casos, o *hash* é adicionado ao `block_hashs` do ficheiro. Quando um ficheiro é apagado, `engine_process_delete()` percorre os seus *hashes* e decrementa o `ref_count` de cada bloco. Se algum chegar a zero, significa que nenhum outro ficheiro o referencia — essa posição no *container* fica disponível na `free_list` para ser reutilizada. O mesmo acontece em `engine_process_truncate()` para os blocos que ficam fora do novo tamanho.

3.2.3. Reutilização de Espaço com a `free_list`

Quando um bloco é libertado, a sua localização física (`container_id` e `offset`) fica guardada na `free_list`. Na próxima vez que for necessário escrever um bloco novo, `chunk_store_write()` verifica primeiro esta lista — se tiver entradas disponíveis, reutiliza esse espaço em vez de escrever no fim do *container*. Só quando a lista está vazia é que o sistema avança para novo espaço, e cria um *container* adicional se necessário.

3.2.4. Limitações do Modelo

O modelo funciona em perfeitas condições para o âmbito do projeto, mas tem limitações que seriam relevantes num sistema real. Como os metadados só são guardados no encerramento, um *crash* inesperado perde tudo o que foi feito desde a última vez que o sistema foi montado. Para além disso, os quatro ficheiros em `meta/` são escritos um a seguir ao outro. Se houver uma falha a meio, o estado em disco fica inconsistente. Num sistema real, seria necessário garantir que os metadados são guardados de forma segura mesmo em caso de falha, o que está fora do âmbito deste projeto.

4. Testes e Avaliação Experimental

Esta secção detalha a metodologia empregada na avaliação do sistema de ficheiros com deduplicação, as ferramentas de observabilidade utilizadas, os cenários de teste executados e apresenta os resultados obtidos.

4.1. Metodologia & Observabilidade (eBPF)

A avaliação do sistema de ficheiros com deduplicação exige uma metodologia rigorosa para quantificar as poupanças de espaço e o impacto no desempenho. Utiliza-se um conjunto de ferramentas padronizadas, com a execução de 30 iterações por carga de trabalho, para garantir a reprodutibilidade, estabilidade estatística e a validez dos resultados.

4.1.1. Ferramentas de Geração de Carga (FIO)

A ferramenta *Flexible I/O Tester* (FIO) foi selecionada para gerar as cargas de trabalho. O FIO permite configurar detalhadamente os padrões de I/O, como o tamanho dos blocos, o tipo de operação (leitura, escrita, aleatória ou sequencial) e a duração dos testes. No contexto deste projeto, o FIO foi configurado para respeitar os requisitos de I/O, nomeadamente operações alinhadas e em múltiplos de 4KB.

4.1.2. Observabilidade no Kernel (eBPF)

Para provar as poupanças de espaço no *kernel*, desenvolveu-se uma ferramenta de observabilidade baseada em eBPF (*extended Berkeley Packet Filter*) denominada `phys_io_counter`.

- **Arquitetura:** Consiste num programa eBPF que executa no *kernel* (`.bpf.c`) e uma aplicação em user-space (`.c`) que controla, recolhe e exibe as métricas.
- **Hook eBPF:** O programa eBPF é anexado ao **tracepoint** `block_rq_issue`. Este ponto de instrumentação, localizado na interface do **block device**, permite intercetar e monitorizar todas as operações de I/O antes que atinjam o hardware de armazenamento.
- **Métricas Recolhidas:** O `phys_io_counter` distingue e agrega as operações de leitura e escrita, e contabiliza os bytes totais transferidos e o número de operações. Estes dados permitem calcular o **throughput** (MB/s) e as operações por segundo (OPS/s) independentemente do sistema de ficheiros.
- **Validação:** As métricas de I/O físico reportadas pelo `phys_io_counter` são usadas para validar as poupanças de espaço indicadas pelo sistema de ficheiros (`dedup_fs`) através do seu ficheiro virtual `/stats.txt`.

4.1.3. Automatização dos Testes

A execução dos testes foi orquestrada por *scripts* shell. Estes *scripts* automatizam o processo de montagem/desmontagem dos sistemas de ficheiros com validações de saúde (*health-checks*), a execução das cargas de trabalho do FIO, a limpeza de **Page Caches** do kernel para evitar falsos positivos de memória, e a recolha de dados em formato CSV para análise posterior.

4.2. Resultados e Comparação

Esta secção apresenta os resultados obtidos nos cenários de teste definidos. Os dados provêm da média de 30 iterações, o que permite uma comparação direta e estatisticamente fiável entre o sistema de ficheiros com deduplicação (`dedup_fs`) e a linha de base sem deduplicação (`passthrough_fs`).

4.2.1. Baseline (FUSE Passthrough)

Como linha de base, utiliza-se uma implementação de sistema de ficheiros *FUSE passthrough*. Este retransmite as operações de I/O para o sistema de ficheiros subjacente sem aplicar lógica de deduplicação, e serve para quantificar o ganho de eficiência ou a sobrecarga induzida pela nossa solução.

4.2.2. Cenários de Teste e Análise de Dados

Cenário 1: Zero Duplicação (Sobrecarga Pior Caso)

- **Objetivo:** Medir a sobrecarga mínima induzida pelo **hashing** (SHA-256) e gestão de metadados.
- **Carga de Trabalho:** Escrita de ficheiro aleatório de 512MB (`fio --name=random-write`).
- **Resultados:** Como esperado, o rácio de deduplicação foi de 1.0. O `passthrough_fs` atingiu 149.5 MiB/s, enquanto o `dedup_fs` registou 129.9 MiB/s. Esta quebra de aproximadamente 13% de **throughput** é acompanhada por um aumento da latência média de 24.8 µs para 28.7 µs. Esta degradação representa o custo computacional puro ("overhead") do sistema quando não existem dados repetidos para compensar o esforço do CPU.

Cenário 2: Alta Duplicação (Clones de Ficheiros)

- **Objetivo:** Avaliar a deduplicação ao nível do ficheiro e o impacto da fragmentação lógica na leitura.
- **Carga de Trabalho:** Criação de 1 ficheiro fonte de 128MB e 3 cópias, seguida de leitura total.
- **Resultados:** A escrita sofreu uma ligeira quebra consistente com o cenário 1 (10%). No entanto, na leitura, o `dedup_fs` registou 65.6 MiB/s contra os 112.4 MiB/s do baseline. Isto evidencia a penalização da "indireção": o kernel necessita de consultar as tabelas de metadados e dar saltos físicos para reconstruir os ficheiros partilhados, o que quebra a sequencialidade de leitura.

Cenário 3: Alta Duplicação (Blocos Repetidos Internamente)

- **Objetivo:** Teste de **stress** à deduplicação com dados massivamente repetitivos.
- **Carga de Trabalho:** Escrita de 512MB com o uso constante do mesmo bloco de 4KB.
- **Resultados:** O ápice da nossa arquitetura. O `dedup_fs` venceu o baseline em velocidade absoluta (157.3 MiB/s vs 153.0 MiB/s) e processou mais de 40.000 IOPS. O rácio de deduplicação foi de 131.072x. Como o sistema detetou as colisões imediatamente, os dados não chegaram a ser enviados para o disco. Isto traduziu-se numa melhoria da latência média para 23.7 µs (face aos 24.4 µs nativos), o que resulta na atualização de referências diretamente na memória RAM em tempo real.

4.2.3. Apresentação dos Resultados

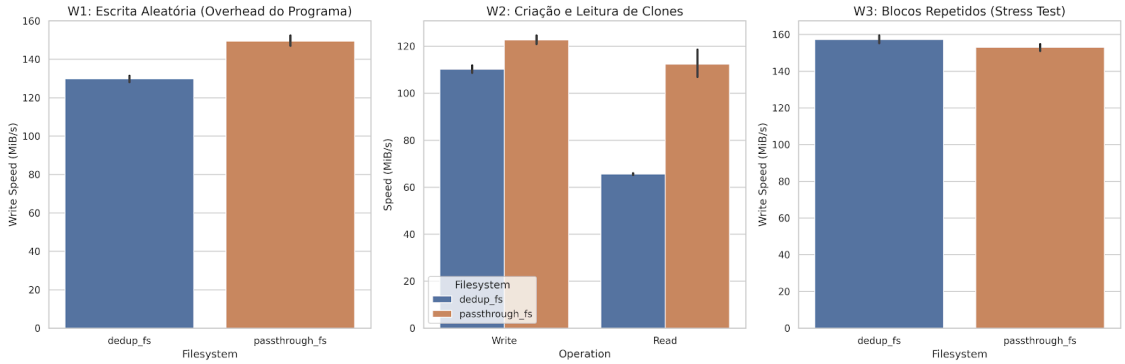


Figura 1: Comparação de Desempenho: Overhead do Programa (W1), Impacto de Leitura (W2) e Ganhos em Stress Test (W3).

Análise de Desempenho Interno (Flamegraph):

Para dissecar o impacto registado no Cenário 1 (Overhead do Programa), recorreu-se à ferramenta `perf` do Linux para fazer o **profiling** do processo `dedup_fs` sob carga de escrita. Os dados capturados foram visualizados num Flamegraph, o que permite uma análise precisa da distribuição do tempo de CPU e a identificação dos **hotspots** computacionais em espaço de utilizador (**user-space**).

A análise revelou que o esforço divide-se em três eixos principais, e justifica a quebra de desempenho face ao modelo nativo:

1. **Lógica de Deduplicação e I/O Físico (50%):** A rotina central `dedup_write` consome metade dos ciclos da aplicação. Esta divide-se na componente criptográfica (`calculate_sha256`), que representa 17.7% do tempo total, e na componente de armazenamento efetivo (`chunk_store_write`), que consome 19.5% a realizar chamadas `pwrite` para o sistema subjacente.
2. **Comunicação FUSE - Receção (29%):** Uma fatia substancial de 29.3% do tempo é gasta unicamente na função `read` (que culmina em `fuse_dev_read`), o que representa o tempo que o daemon passa a ingerir os pedidos de I/O provenientes do kernel.
3. **Comunicação FUSE - Resposta (14%):** O envio do sinal de confirmação para o kernel (`fuse_reply_write`) exige 14.3% adicionais do tempo de CPU.

Conclui-se que quase 43% do tempo de CPU é acrescido devido à utilização do FUSE (leitura e resposta de pedidos via `/dev/fuse`), enquanto a mecânica real de deduplicação (hashing) impõe uma penalização direta de apenas cerca de 18%. Estes dados comprovam empiricamente a origem da quebra de **throughput** registada no Workload 1.

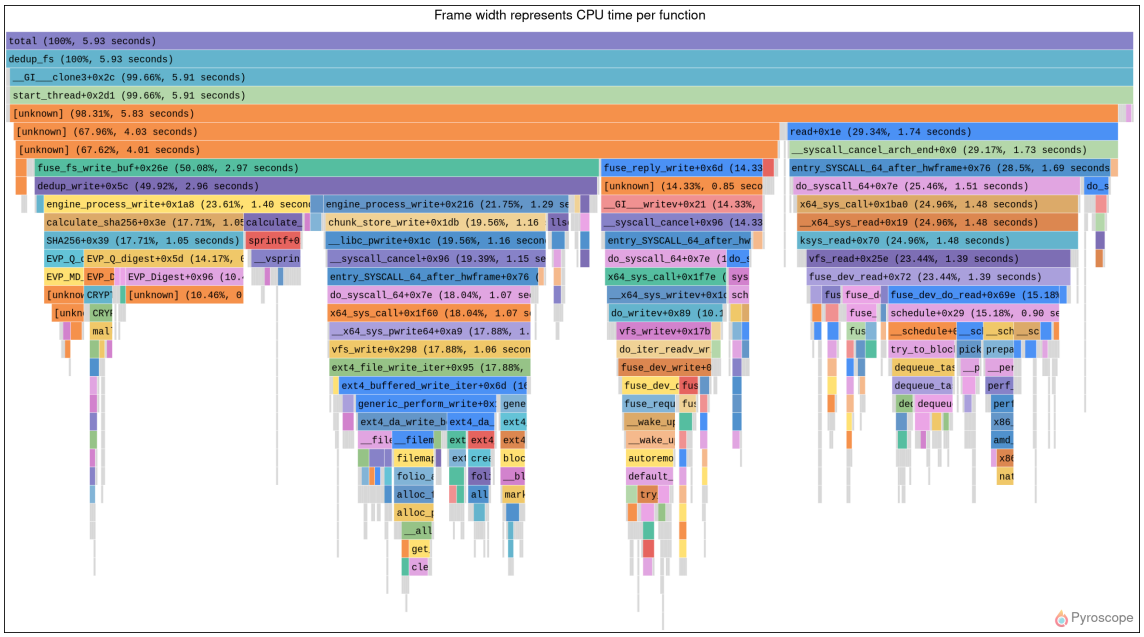


Figura 2: Flamegraph que destaca o custo criptográfico e o overhead de comunicação FUSE (receção e resposta de I/O).

Cenário de Teste	Métrica	dedup_fs	passthrough_fs	Diferença (%)
1: Aleatório (W1)	Rácio Deduplicação	1.00x	1.00x	0.0 %
1: Aleatório (W1)	Throughput Escrita	129.9 MiB/s	149.5 MiB/s	−13.1 %
1: Aleatório (W1)	Latência Média	28.66 µs	24.81 µs	+15.5 %
2: Clones (W2)	Rácio Deduplicação	4.00x	1.00x	+300 %
2: Clones (W2)	Throughput Escrita	110.3 MiB/s	122.7 MiB/s	−10.1 %
2: Clones (W2)	Throughput Leitura	65.6 MiB/s	112.4 MiB/s	−41.6 %
3: Stress (W3)	Rácio Deduplicação	131072.00x	1.00x	Máximo
3: Stress (W3)	Throughput Escrita	157.3 MiB/s	153.0 MiB/s	**+ 2.8 %**
3: Stress (W3)	Latência Média	23.70 µs	24.35 µs	**- 2.7 %**

Tabela 2: Resumo das Médias de Desempenho (30 Iterações) entre dedup_fs e passthrough_fs

5. Conclusão

O desenvolvimento deste projeto permitiu a implementação, observabilidade e avaliação rigorosa de um sistema de ficheiros com deduplicação **inline** ao nível do bloco, que opera no espaço de utilizador (user-space) através da interface FUSE.

No desenho da arquitetura, a escolha de particionar os dados em **chunks** fixos de 4KB, aliada ao algoritmo de **hashing** criptográfico SHA-256, revelou-se uma fundação sólida para a identificação de blocos redundantes. A manutenção das estruturas de dados, do mapeamento de ficheiros para **hashes** lógicos e da tabela de contagem de referências físicas, permitiu a partilha transparente de blocos entre ficheiros distintos, sem comprometer a integridade da árvore do sistema.

A introdução da tecnologia eBPF no projeto constituiu um marco fundamental na validação da solução. Através da monitorização direta ao nível do **kernel** (`block_rq_issue`), foi possível contornar a abstração das **Page Caches** do Linux e comprovar de forma empírica que a deduplicação lógica se traduz, de facto, numa poupança real e dramática de I/O físico para o hardware de armazenamento.

A avaliação de desempenho colocou em perspetiva a clássica dualidade de compromisso (**trade-off**) associada a sistemas de deduplicação e ao uso do FUSE:

- **O Custo Arquitetural:** A análise de Flamegraphs evidenciou que a quebra base de desempenho em cargas de trabalho não-repetitivas, de cerca de 13%, provém, em grande parte, das trocas de contexto (**context-switches**) exigidas pela interface FUSE e do processamento computacional inerente ao cálculo criptográfico e gestão de indireção. Adicionalmente, verificou-se uma quebra expressiva nas velocidades de leitura, até 41%, devido à perda de sequencialidade dos blocos físicos dispersos no disco.
- **O Ganho Operacional:** Em oposição, perante cenários com elevada densidade de duplicação, o sistema isolou na perfeição o estrangulamento do disco físico. Ao processar dados repetidos unicamente na memória RAM através da atualização de referências lógicas, o sistema suprimiu as escritas físicas e atingiu um rendimento que superou uma montagem nativa não-deduplicada.

Em suma, o trabalho cumpre com sucesso os objetivos propostos na disciplina de Tecnologias de Sistemas Operativos. O `dedup_fs` demonstra que, embora os sistemas **User-Space** acarretem custos acrescidos de comunicação com o **kernel**, a aplicação inteligente de algoritmos de deduplicação compensa estas penalizações em ambientes focados em **backups**, virtualização ou armazenamento intensivo de dados redundantes, o que converte o esforço computacional do CPU em poupança massiva de I/O físico e espaço em disco.

6. Referências Bibliográficas

[1] Paulo J, «Material de Apoio da Unidade Curricular de Tecnologias de Sistemas Operativos». 2026.

[2] The Linux Kernel Development Community, «FUSE: Filesystem in Userspace». [Em linha]. Disponível em: https://www.kernel.org/doc/html/latest/filesystems/fuse.html

[3] eBPF Foundation, «eBPF - Introduction, Tutorials & Community Resources». [Em linha]. Disponível em: https://ebpf.io/

[4] J. Axboe, «Fio - Flexible I/O Tester Documentation». [Em linha]. Disponível em: https://fio.readthedocs.io/en/latest/fio_doc.html